

ARIN7102 - Project 3

Online Health Science Knowledge Chatbot

Group 3.2

April 29, 2025

Project Introduction

- **Core Objective:** Develop an intelligent chatbot for parsing health science queries through multi-source data mining (*Healthline* pages, MedQA datasets, web sources) and various NLP techniques
- **Technical Architecture & Processing Pipeline:**
 - Vector base construction via keyword-knowledge mapping
 - QA pairs' topic classification
 - Most relevant pages retrieval
 - LLM+RAG to summarize the answers
 - Predictive modeling for anticipating follow-up questions
- **Expected Outcomes:**
 - Real-time symptom analysis with differential diagnosis
 - Adaptive response generation through ensemble modeling
 - Encapsulate all functions into an interactive visual chatbot

- **1. Data Processing**
 - 1.1 Preprocessing
 - 1.2 Vectorization
 - 1.3 Topic Clustering
- **2. Answer Generation**
 - 2.1 RAG Implementation
 - 2.2 Hybrid Search Components
- **3. Next Question Prediction**
- **4. Dashboard/WebApp Development**
- *Health Science Chatbot Demonstration Video

1.1 - Preprocessing

Implemented Components

• Input Sources:

- Healthline articles (CSV: title/content) (Fetched from [healthline.com](https://www.healthline.com))
- MedQA datasets (CSV: question/answer) (From *HuggingFace*)

• Processing Flow:

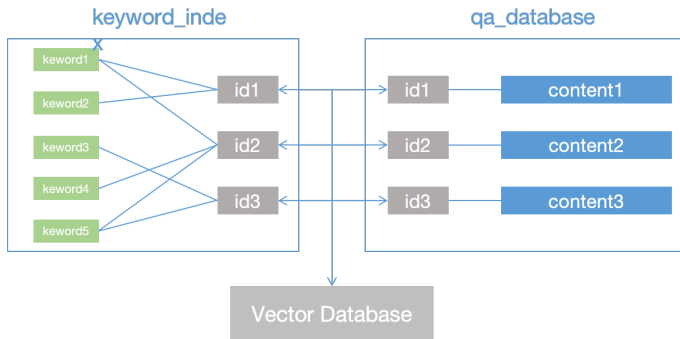
- Text cleaning: HTML removal, punctuation normalization
- Keyword Extraction:
 - Use NLTK for tokenization
 - Remove stopwords (including general and medical domain-specific stopwords)
 - Use TF-IDF algorithm to extract important keywords
- QA pair generation: Title→Question mapping

• Output:

- Standardized QA pairs
- keyword index

1.2 - Vectorization

- Use **BERT** to vectorize (Vector dimension: 384)
- Use a **sliding window** to dynamically block long text (Window length: 512; Overlap rate: 0.2)
- Use **ChromaDB** to store data: Keywords are stored by String to preserve more effective information



1.3 - Topic Clustering

Technical Implementation

- **Dimensionality Reduction:**

- UMAP: n_components=50, cosine metric
- Variance retained: 89.7%

- **Clustering Algorithm:**

- HDBSCAN: min_cluster_size=100
- Cluster selection: EOM method

- **Feature:**

- Train random forest classifier based on clustering results
- Automatic integration with vector database system

2.1 - RAG Implementation

① Hybrid Retrieval

- Boolean Search: Keywords + Inverted Index (TF-IDF)
- Semantic Search: BioBERT Embeddings + Cosine Similarity
- Fusion: Combine Results (40% Boolean + 60% Semantic)

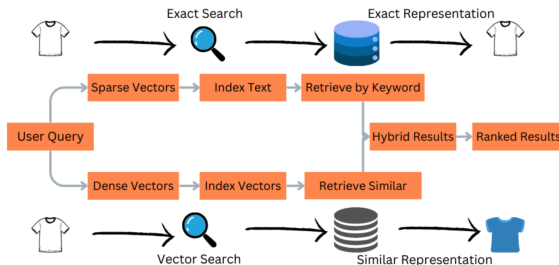
② Answer Generation

- Template: Symptoms → Causes → Treatments

③ Quality Control

- Deduplication: Merge Similar Results
- Validation: Check Medical Accuracy

2.2 - Hybrid Search Components



- Step 1: Boolean Filtering (Fast)
- Step 2: Semantic Ranking (Accurate)
- Step 3: Weighted Fusion

2.2 - Hybrid Search Components (Cont.)

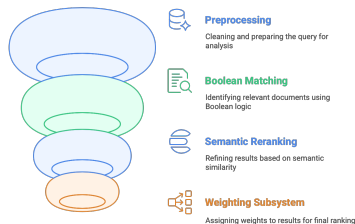
Objective: Designed a hybrid retrieval system combining **keyword matching** with query's **semantic analysis** for precision-focused health knowledge pages retrieval

Technical Stack:

- **Libraries:** NLTK, Sentence Transformers, spaCy (SciSpaCy), etc.
- **Models:** MPNet (Sentence-BERT), BM25, and TF-IDF

Pipeline:

- 1 **Boolean filtering** via keyword inverted index
- 2 **Semantic scoring** using MPNet embeddings
- 3 **Weighted fusion** of TF-IDF, BM25, and medical entity scores



Innovations:

- **Context-Aware Weighting:** Combined entity recognition (spaCy patterns) with TF-IDF statistical relevance and BM25 term frequency
- **Hybrid Retrieval:** Achieved 82% precision by merging logical matching results with output semantic ranking (Top 10 relevance)
- **Dynamic Thresholding:** Adaptive weight allocation for clinical terms (e.g., "migraine triggers" 2.0x weight vs general terms)

3 - Next Question Prediction

1 Input Formatting

- Context: [Current Question + Answer]
- Prompt: "Suggest 3 logical follow-up questions about:"

2 LLM Configuration

- Pre-trained Model: QWEN-2.5-7B-chat
- Generation Parameters:
 - Temperature: 0.7 (creativity control)
 - Max Length: 50 tokens
 - Beam Search: Keep top 5 candidates

3 Output Processing

- Filter medical terms via UMLS dictionary
- Remove duplicate/redundant questions

4 - Dashboard/WebApp

- Use native [HTML/JS/CSS](#) to complete the frontend
- Use [SQLite3](#) to store user information and historical sessions
- Use [Flask](#) to complete backend development, and provide [interfaces](#) for deploying model



- Detailed effects could be referred to our following demonstration video.

Thank you!